

Spring MVC Test

@SpringBootTest

- Cette annotation permet d'instancier un ApplicationContext complet pour fin de test.
- Utilisé avec l'annotation @AutoConfigureMvc qui permet d'injecter un objet MockMvc
- l'objet MockMvc permet par la suite d'interroger les controleurs de l'application
- On peut par la suite utiliser des 'MockMvcResultMatchers' pour extraire et vérifier que l'on a les bonnes données

@SpringBootTest

```
@SpringBootTest
@AutoConfigureMockMvc
public class TestingWebApplicationTest {

    1 usage
    @Autowired
    private MockMvc mockMvc;

    no usages  Dave Syer +1 *
    @Test
    public void shouldReturnDefaultMessage() throws Exception {
        this.mockMvc.perform(get(urlTemplate: "/"))
            .andDo(print())
            .andExpect(status().isOk())
            .andExpect(content().string(containsString(substring: "Hello, World")));
    }
}
```


@WebMvcTest

- Cette annotation permet de faire la même chose que @SpringBootTest mais sans la lourdeur de tout chargé ('loadé') le ApplicationContext
- Cette habilité est appelé 'slicing'. Ainsi, l'environnement de test ne charge que les dépendances nécessaires à l'exécution du test ce qui accélère grandement l'exécution du test.
- Il est donc largement favorable d'utiliser cette annotation plutôt que @SpringBootTest

@WebMvcTest

```
// too heavy to load entire spring context, uses @WebMvcTest
//@SpringBootTest
//@AutoConfigureMockMvc

@WebMvcTest(MyController.class)
public class MyControllerTest {

    @Autowired
    private MockMvc mvc;

    @Test
    public void testHello() throws Exception {

        mvc.perform(get("/endpoint")
            .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("hello world"));

    }
}
```

@WebMvcTest

- la méthode 'perform()' permet d'accéder au 'endpoint' de notre webservice qui est implanté via un '@RestController' dans notre application SpringBoot
- Il est possible de tester tous les verbes HTTP (GET, POST, DELETE, PUT)
- <https://howtodoinjava.com/spring-boot2/testing/spring-boot-mockmvc-example/>

@MockMvcResultMatchers

- Fournis les ‘assertions’ pour vérifier le ‘code status’, le contenu ou les ‘headers’ de la réponse.
- `MockMvcResultMatchers.status().isOk()`
- `MockMvcResultMatchers.content().string(“expected response”)`
- `MockMvcResultMatchers.header().exists(“Custom-Header”)`
- `MockMvcResultMatchers.header().string(“Custom-Header”, “SomeValue”)`

MockMvcResultMatchers.jsonPath()

- le ResultMatcher jsonPath permet d'extraire du contenu du Json de retour pour fin d'assertion. Voici quelques liens qui vous donneront plus de détail.
- <https://goessner.net/articles/JsonPath/>
- <https://mkyong.com/spring-boot/testing-json-in-spring-boot/>
- <https://www.diffblue.com/blog/java/software%20development/testing/spring-boot-test-slices-overview-and-usage/>
- <https://rieckpil.de/guide-to-testing-spring-boot-applications-with-mockmvc/>
- <https://rieckpil.de/testing-your-json-serialization-with-json-test/>